

Residual Multi-Tree Genetic Programming for Joint Adaptation of State Transition and Pheromone Update Rules in Ant Colony Optimization

First Author, Second Author, and Third Author

Abstract—Ant colony optimization uses a state transition rule and a pheromone update rule. Both rules are usually designed by experts. Genetic programming can automate this design process. However, direct replacement of a complete rule can remove useful structure from the original algorithm. Existing work has also focused mainly on state transition. This study proposes residual multi-tree genetic programming for ant colony optimization, called RMTGP-ACO. One strongly typed tree adjusts transition desirability. A second strongly typed tree redistributes pheromone deposits. Both adjustments are bounded. The original algorithm is recovered when both trees return zero. The pheromone tree also preserves the deposit budget of each reinforcement event. The method is applied to Ant System, Ant Colony System, and MAX-MIN Ant System. The current pilot uses uniform TSP100 instances for training. It uses three independent genetic programming runs for each ACO variant. The selected programs are tested on uniform TSP50, TSP100, TSP500, and TSP1000. Full RMTGP-ACO reduces the mean reference gap in all 12 completed variant-scale comparisons. The reduction ranges from 0.0736 to 7.8461 percentage points. A fused CUDA implementation gives a 14.626-fold single-GPU speedup over a 16-thread CPU reference on the registered benchmark. The present evidence is preliminary. The residual, terminal-set, function-set, and single-tree ablations are still in progress.

Index Terms—Ant colony optimization, genetic programming, hyper-heuristic, multi-tree genetic programming, residual learning, traveling salesman problem.

I. INTRODUCTION

ANT colony optimization (ACO) constructs solutions from a pheromone model and problem-specific heuristic information [1], [2]. Its behavior depends on two central components. The state transition rule selects the next solution component. The pheromone update rule changes the sampling model after each iteration. These rules are effective, but they are usually fixed by human design.

Genetic programming (GP) can evolve executable rules [3], [4]. It can therefore reduce the need for manual formula design. Previous GP-ACO work evolved complete state transition rules [5]. That study showed that evolved rules can transfer across traveling salesman problem (TSP) sizes and distributions. It also showed that global terminals can be useful. However, a complete replacement changes the full transition model. It does not preserve the original balance between pheromone and heuristic information. It can also produce unstable score scales.

Pheromone adaptation creates a second design problem. Earlier work used strongly typed GP to evolve pheromone

strategies [6]. This direction is important, but joint transition and pheromone adaptation is difficult. The two rules act at different points in ACO. Their tensors also have different meanings and shapes. A shared untyped tree would allow invalid combinations. Two unrestricted trees would also receive more representational capacity than one tree.

We propose residual multi-tree genetic programming for ACO, or RMTGP-ACO. The method uses one transition tree and one pheromone tree. The trees have separate strong types. The transition tree multiplies the original desirability by a bounded residual factor. The pheromone tree redistributes a fixed deposit budget over the edges that the original ACO would reinforce. The method retains the main semantics of Ant System (AS), Ant Colony System (ACS), and MAX-MIN Ant System (MMAS) [1], [7], [8].

This study makes four technical contributions.

- It defines a bounded residual interface for ACO state transition.
- It defines an event-wise pheromone residual that preserves support and total deposit.
- It introduces a strongly typed two-tree representation with a shared node budget.
- It provides a fused GPU evaluator for the nested GP-ACO computation.

The current results form a three-run pilot. They answer whether the complete method improves the original ACO on uniform Euclidean TSP. They do not yet establish which design component causes the improvement. We state this boundary throughout the paper.

II. BACKGROUND AND PROBLEM DEFINITION

A. Euclidean Traveling Salesman Problem

Let $V = \{1, \dots, n\}$ be a set of cities. Each city i has a coordinate $c_i \in \mathbb{R}^2$. The distance between cities i and j is

$$d_{ij} = \|c_i - c_j\|_2. \quad (1)$$

A tour is a permutation $\pi = (\pi_1, \dots, \pi_n)$. Its length is

$$L(\pi) = \sum_{q=1}^{n-1} d_{\pi_q, \pi_{q+1}} + d_{\pi_n, \pi_1}. \quad (2)$$

The objective is to minimize $L(\pi)$.

Each dataset row contains a reference tour. We recompute its length in float64. The available files do not provide a global optimality certificate for every tour. We therefore use

the term *reference length*, denoted by L_i^{ref} . The primary quality measure is

$$g_i(\theta) = 100 \frac{L_i(\theta) - L_i^{\text{ref}}}{L_i^{\text{ref}}}. \quad (3)$$

We call g_i the reference gap.

B. Baseline ACO State Transition

Let τ_{ij} be the pheromone value of edge (i, j) . Let

$$\eta_{ij} = \frac{1}{d_{ij} + \epsilon_d} \quad (4)$$

be its heuristic value. At construction step q , ant a is at city i . Its feasible set is $F_{a,q}$. The baseline desirability of city $j \in F_{a,q}$ is

$$s_{aij}^0 = \tau_{ij}^\alpha \eta_{ij}^\beta. \quad (5)$$

AS and MMAS sample from the normalized score. ACS uses the pseudorandom proportional rule. It selects the maximum score with probability q_0 . Otherwise, it samples from the normalized score. All variants use a candidate list. If that list has no feasible city, the implementation uses the full feasible set.

C. Baseline Pheromone Update

AS evaporates all pheromone and reinforces the tours of all ants. For ant a , the baseline deposit on a tour edge is $1/L_a$. ACS applies its local update during construction. It then applies a global update to the global-best tour. MMAS reinforces one scheduled tour. The schedule uses iteration-best tours and restart-best tours. MMAS also applies dynamic pheromone bounds and restart logic.

Our implementation follows the algorithmic semantics of ACOTSP 1.03 [9]. All three variants use 32 ants and 500 iterations in this study. Local search is disabled.

D. Previous GP-ACO Baseline

The previous study evolved one complete state transition expression [5]. We denote a matched implementation of that approach by Legacy-GP. Its terminals are raw pheromone, raw distance, mean pheromone, mean distance, problem size, and feasible-set size. It uses arithmetic primitives and a protected division that returns one when the denominator is close to zero.

The present study retrains Legacy-GP under the current data, ACO budget, and random schedules. Published numerical results are not copied into the current paired tables. This rule prevents a comparison between incompatible budgets.

III. RESIDUAL MULTI-TREE GP-ACO

A. Overview

An RMTGP-ACO individual contains two programs:

$$\theta = (T_{\text{tr}}, T_{\text{ph}}). \quad (6)$$

The transition program acts during solution construction. The pheromone program acts after the colony tours are available. Fig. 1 shows the data flow.

B. Strongly Typed Representation

The implementation uses strongly typed GP [10]. It defines two marker types.

$$\text{TrField} \in \mathbb{R}^{B \times M \times K}, \quad (7)$$

$$\text{PhField} \in \mathbb{R}^{B \times R \times n}. \quad (8)$$

Here, B is the instance batch size. M is the number of ants. K is the active candidate-set width. R is the number of deposit events.

Every transition terminal returns a TrField. Every transition function maps one or two TrField values to a TrField. The pheromone primitive set follows the same rule for PhField. The two marker types are different Python classes. DEAP therefore rejects a subtree exchange between the two roles [11].

The two trees share one fitness. They also share a total size limit:

$$N_{\text{tr}} + N_{\text{ph}} \leq 31. \quad (9)$$

An inactive zero tree has zero effective nodes. Each nonzero tree has a separate maximum of 31 syntax nodes. Each tree also has a maximum depth of five. Equation (9) is the active constraint in the main study. It prevents a two-tree individual from receiving twice the capacity of a one-tree individual.

C. Transition Residual

The transition tree receives the terminal field x_{aij} . Its raw output is

$$u_{aij} = T_{\text{tr}}(x_{aij}). \quad (10)$$

The output changes the baseline score in (5):

$$r_{aij} = \tanh(u_{aij}), \quad (11)$$

$$m_{aij} = 1 + \gamma_{\text{tr}} r_{aij}, \quad (12)$$

$$\tilde{s}_{aij} = s_{aij}^0 m_{aij}, \quad (13)$$

where $\gamma_{\text{tr}} = 1/3$. The final probability is

$$p_{aij} = \frac{\tilde{s}_{aij}}{\sum_{\ell \in F_{a,q}} \tilde{s}_{ail}}. \quad (14)$$

This interface has four useful properties. First, a zero tree gives $m_{aij} = 1$. It exactly recovers the baseline ACO. Second, $m_{aij} \in [2/3, 4/3]$. The score remains positive. Third, infeasible edges remain infeasible. Fourth, for two feasible cities j and k ,

$$\frac{m_{aij}}{m_{aik}} \in [1/2, 2]. \quad (15)$$

The tree can change a local preference, but it cannot create an unbounded score ratio.

D. Pheromone Residual

A deposit event is

$$r = (\pi_r, L_r, w_r). \quad (16)$$

π_r is a source tour. L_r is its length. w_r is the baseline weight. For each edge e in π_r , the baseline deposit is

$$D_{r,e}^0 = \frac{w_r}{L_r}. \quad (17)$$

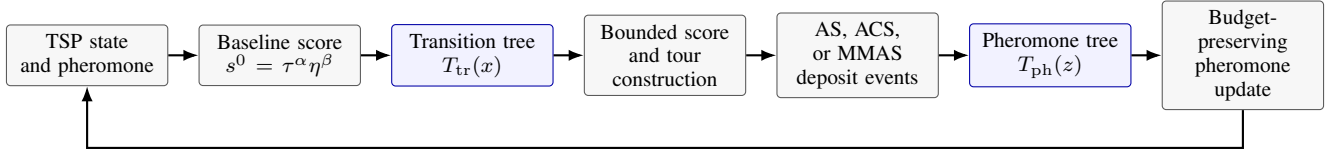


Fig. 1. RMTGP-ACO data flow. The blue boxes are evolved programs. The other boxes retain the semantics of the selected ACO variant.

The total event budget is

$$B_r = \sum_{e \in E(\pi_r)} D_{r,e}^0 = \frac{nw_r}{L_r}. \quad (18)$$

The pheromone tree produces

$$v_{r,e} = T_{ph}(z_{r,e}), \quad (19)$$

$$h_{r,e} = 1 + \gamma_{ph} \tanh(v_{r,e}), \quad (20)$$

where $\gamma_{ph} = 1/3$. The final deposit is

$$\tilde{D}_{r,e} = B_r \frac{D_{r,e}^0 h_{r,e}}{\sum_{f \in E(\pi_r)} D_{r,f}^0 h_{r,f}}. \quad (21)$$

The zero tree recovers the baseline deposit. The reinforced edge set is unchanged. The event budget is also conserved:

$$\sum_{e \in E(\pi_r)} \tilde{D}_{r,e} = B_r. \quad (22)$$

AS creates one event for each ant. ACS creates one global-best event. MMAS creates one event from its original update schedule. The residual only redistributes each event. It does not modify ACS local updates, MMAS bounds, or MMAS restart decisions.

E. Terminal Normalization

Many raw ACO values depend on the problem size and iteration. The main terminal set uses bounded relative values. For a vector y and its active mask, define

$$\text{stdrel}(y) = \tanh\left(\frac{y - \text{mean}(y)}{\text{std}(y) + 10^{-8}}\right). \quad (23)$$

The mean and standard deviation are computed over the last active dimension. Masked entries receive zero.

The local-search extension adds an eighth pheromone terminal, LSGain . Its strong type is PhField , and its logical shape is $[B, R, n]$. For source tour r , it is

$$2 \text{clip}\left(\frac{L_r^{\text{pre}} - L_r^{\text{post}}}{\max(L_r^{\text{pre}}, \epsilon)}, 0, 1\right) - 1.$$

The inputs are the current tour and its lengths before and after local search. No reference solution is used. The resulting $[B, R]$ value is broadcast over the n source-tour edges. It is fixed at -1 when local search is disabled. The main study retains the seven terminals in Table II. Both training conditions in the local-search extension use the same eight-terminal grammar.

Both primitive sets also contain the constants $-1, -0.5, 0, 0.5, 1$. They contain one ephemeral random constant sampled from $U[-1, 1]$. An ERC mutation adds Gaussian noise with standard deviation 0.1 . The result is clipped to $[-1, 1]$. The proposed Full-F1 profile uses all terminals in Tables I and II. It uses the F1 function profile.

F. Function Sets and Numerical Protection

Table III gives the two function profiles. All operations are element-wise. F1 extends F0. Protected division is

$$\text{PDIV}(x, y) = \frac{xy}{y^2 + 10^{-6}}. \quad (24)$$

This form is continuous at $y = 0$. After each primitive, NaN is replaced by zero. Positive and negative infinity are replaced by 10 and -10 . Every result is then clipped to $[-10, 10]$. The external residual interface applies another $\tanh$.

Legacy-GP uses ADD, SUB, MUL, NEG, and PDIV1. Its protected division returns x/y when $|y| > 10^{-6}$. It returns one otherwise. Matched-Replace uses the Full terminals and F1. Its raw transition output becomes a positive score through

$$s_{aij} = \text{softplus}(\text{clip}(u_{aij}, -20, 20)) + \epsilon. \quad (25)$$

G. Evolutionary Search

The initial joint population contains four modes. Ten percent are exact baseline sentinels. Twenty percent activate only the transition tree. Twenty percent activate only the pheromone tree. The remaining 50 percent activate both trees. Active trees use ramped half-and-half initialization with depths from two to four.

Elitism keeps the best ten individuals. The remaining offspring use crossover with probability 0.80 , mutation with probability 0.15 , and reproduction with probability 0.05 . Tournament size is four. Crossover selects one active role and applies typed one-point crossover within that role. Mutation selects one active role. It uses subtree mutation with probability 0.50 , node replacement with probability 0.30 , and ERC perturbation with probability 0.20 . An offspring that violates the depth or node limit is replaced by its valid parent.

Structural hashes avoid repeated evaluation within one generation. The GPU evaluator also groups programs with identical compiled behavior. These operations do not change the fitness.

For a training set $\mathcal{I}$ and ACO seed set $\mathcal{S}$, fitness is

$$F(\theta) = \frac{1}{|\mathcal{I}||\mathcal{S}|} \sum_{i \in \mathcal{I}} \sum_{s \in \mathcal{S}} g_{i,s}(\theta). \quad (26)$$

The objective minimizes the absolute reference gap. It does not minimize a baseline-relative value. It contains no parsimony penalty. Program size is used only to break defined ties. For reporting, we separately define

$$\Delta g = g_{\text{method}} - g_{\text{ACO}}. \quad (27)$$

A negative Δg favors the learned method.

TABLE I

TRANSITION TERMINAL DEFINITIONS. ALL OUTPUTS ARE BROADCAST TO TrField. CORE USES THE FIRST FOUR TERMINALS. FULL USES ALL EIGHT TERMINALS.

Terminal	Definition	Meaning and calculation domain
RTau	$\text{stdrel}(\log(\max(\tau_{ij}, \epsilon)))$	Relative log pheromone. Statistics use the active feasible set of one ant at one construction step.
REta	$\text{stdrel}(\log(\max(\eta_{ij}, \epsilon)))$	Relative log heuristic. Statistics use the same active feasible set.
BaseConf	$\tanh(\log(p_{aij}^0 + \epsilon) + \log k_{a,q})$	Local confidence of the baseline distribution. $k_{a,q} = F_{a,q} $. A uniform baseline gives zero for every feasible city.
DistRank	$1 - 2(r_{aij}^d - 1)/(k_{a,q} - 1)$	Distance rank inside the active feasible set. Rank one maps to 1. The farthest city maps to -1 . The value is zero when $k_{a,q} = 1$.
Entropy	$2H(p_{ai}^0)/\log k_{a,q} - 1$	Normalized entropy of the baseline distribution. The value is broadcast across candidates. It is set to -1 when $k_{a,q} = 1$.
ConstructProg	$2q/\max(n - 1, 1) - 1$	Progress within the construction of one tour. q is the current construction step.
ACOProg	$2(t - 1)/\max(T - 1, 1) - 1$	Progress within the ACO run. t is the current ACO iteration and T is the iteration budget.
Stagnation	$2\min(s_t/T, 1) - 1$	Progress since the last global-best improvement. The value is broadcast across ants and candidates.

TABLE II

PHEROMONE TERMINAL DEFINITIONS. ALL OUTPUTS ARE PhField. CORE USES EDGEETA, EDGETAU, NNRank, AND SOURCEQUALITY. FULL USES ALL SEVEN TERMINALS.

Terminal	Definition	Meaning and calculation domain
EdgeEta	$\text{stdrel}(\log \eta_{uv} - (\mu_u^\eta + \mu_v^\eta)/2)$	Relative heuristic value of event edge (u, v) . μ_u^η is the mean log heuristic in the candidate neighborhood of u . The final normalization is inside the source tour.
EdgeTau	$\text{stdrel}(\log(\max(\tau_{uv}, \epsilon)))$	Relative pheromone on a source-tour edge. Statistics are computed inside one deposit event.
NNRank	$\frac{1}{2}[\nu(r_{uv}) + \nu(r_{vu})]$	Symmetric nearest-neighbor rank. Here $\nu(r) = 1 - 2(r - 1)/\max(n - 2, 1)$. Full neighbor ranks are used.
ColonyFreq	$2c_e/M - 1$	Frequency of edge e in the current colony. c_e is the number of ant tours that contain the undirected edge.
SourceQuality	$\tanh((\bar{L} - L_r)/(\sigma_L + \epsilon))$	Quality of the source tour relative to the current colony. The value is broadcast to all edges of the event.
ACOProg	$2(t - 1)/\max(T - 1, 1) - 1$	Progress within the ACO run.
Stagnation	$2\min(s_t/T, 1) - 1$	Progress since the last global-best improvement.

TABLE III
GP FUNCTION PROFILES

Profile	Binary functions	Unary functions
F0	ADD, SUB, MUL, PDIV	NEG
F1	F0, MIN, MAX	NEG, ABS

IV. EXPERIMENTAL METHODOLOGY

A. Research Questions

The study is organized around seven questions.

- RQ1: Does Full-F1 RMTGP-ACO reduce reference gap relative to the original ACO?
- RQ2: Is residual transition adaptation better than matched full replacement?
- RQ3: Does a two-tree individual outperform independently trained one-tree methods under an equal total node budget?
- RQ4: What are the effects of the terminal profile and function profile?

RQ5: Does joint coadaptation matter after the two trees are trained?

RQ6: Does a TSP100-trained individual transfer across problem sizes and distributions?

RQ7: How much speedup does the fused evaluator provide?

B. Data

The training pool contains 1,280,000 uniform TSP100 instances. It consists of ten files with 128,000 instances each. A frozen schedule selects 32 different instances per generation. One GP run therefore sees $32 \times 50 = 1,600$ different training instances. The full pool is much larger than the consumed subset. This design supports independent schedules for additional GP runs.

The validation file contains 1,280 uniform TSP100 instances. Each GP run uses 32 instances for selection and 32 disjoint instances for the non-inferiority gate. The completed test sets contain 1,280 uniform TSP50 instances, 1,280 uniform TSP100 instances, 128 uniform TSP500 instances, and

Algorithm 1 One RMTGP-ACO simulation

Require: TSP instance, ACO variant, T_{tr} , T_{ph}

- 1: Initialize pheromone and variant-specific search state
- 2: **for** $t = 1, \dots, T$ **do**
- 3: **for** each construction step q **do**
- 4: Build the active candidate set
- 5: Compute baseline score $s^0 = \tau^\alpha \eta^\beta$
- 6: Build only the terminals used by T_{tr}
- 7: Apply the bounded residual in (13)
- 8: Select cities with the original variant rule
- 9: Apply the ACS local update when required
- 10: **end for**
- 11: Compute all tour lengths and update best states
- 12: Build AS, ACS, or MMAS deposit events
- 13: Build only the terminals used by T_{ph}
- 14: Apply the budget residual in (21)
- 15: Apply variant evaporation, bounds, and restart logic
- 16: **end for**
- 17: **return** Best tour; recompute its length in CPU float64

Algorithm 2 GP training and champion selection

Require: Frozen training and validation schedules

- 1: Load or create immutable baseline ACO archives
- 2: Initialize 100 typed two-tree individuals
- 3: **for** $h = 1, \dots, 50$ **do**
- 4: Load 32 scheduled TSP100 training instances
- 5: Use one shared ACO seed for all individuals
- 6: Evaluate unique programs and assign (26)
- 7: Save the top five every five generations
- 8: Create the next population with typed operators
- 9: **end for**
- 10: Screen unique checkpoints on 32 selection instances with one seed
- 11: Re-evaluate the best five with three paired seeds
- 12: Find the minimum mean gap
- 13: Within 0.01 percentage points, select the smaller program
- 14: Audit this candidate on 32 disjoint gate instances
- 15: **return** One selected candidate and one gate-deployed policy

128 uniform TSP1000 instances. TSP1000 is a locked-model supplementary extrapolation set. It is not used during evolution or model selection.

The registered distribution tests contain 128 clustered TSP500 instances, 128 Gaussian TSP500 instances, and 42 TSPLIB instances with $n \leq 500$ [12]. Their current results are pending. All files are identified by a manifest and a SHA-256 checksum. Train, validation, and test coordinates remain separated. Reference-tour edges are never used as GP terminals.

C. Compared Methods and Ablations

Table V maps each method to its scientific purpose. Full-F1 is the proposed main configuration. Core and Full refer to the terminal sets in Tables I and II. F0 and F1 refer to Table III.

TABLE IV
DATA USED BY THE TSP100 PILOT

Role	Partition	Instances
Training pool	TSP100 uniform	1,280,000
Per GP run	32×50 TSP100	1,600
Validation	Selection / gate	32/32
Test	TSP50 uniform	1,280
Test	TSP100 uniform	1,280
Test	TSP500 uniform	128
Test	TSP1000 uniform	128
Pending test	TSP500 cluster / Gaussian	128/128
Pending test	TSPLIB, $n \leq 500$	42

RQ2 uses TR-RGP minus Matched-Replace as its primary estimand. Legacy-GP is a secondary methodological baseline. RQ3 compares Full-F1 with independently trained TR-RGP and PH-RGP. All three methods first use a total node budget of 31. A separate capacity study compares budgets of 31 and 62. RQ4 uses a 2×2 factorial design. It estimates the terminal main effect, the function main effect, and their interaction. RQ5 drops each tree and shuffles independently evolved tree pairs. The post-hoc interventions complement the independent one-tree controls. They do not replace them.

D. ACO and GP Parameters

Table VI gives the common parameters. AS uses $\rho = 0.5$. ACS uses $\rho = 0.1$, $q_0 = 0.9$, and $\xi = 0.1$. MMAS uses $\rho = 0.02$. Its update period is 25. Its p_{best} value is 0.05. Its branch check period is 100. Its branch parameter is 0.05. Its branch threshold is 1.00001. Its restart stagnation limit is 250. ACS applies synchronous local updates.

The GP root seeds are 1001–1003 for AS, 2001–2003 for ACS, and 3001–3003 for MMAS. Final test seeds are derived from a separate root seed of 9001. They are independent of the GP seeds. All methods within a paired comparison share the same instances and ACO seeds.

E. Champion Selection and Reported Policy

Each generation uses one ACO seed. All individuals share that seed and the same 32 instances. This pairing reduces within-generation noise. The baseline ACO values are pre-computed and stored in immutable archives.

The validation procedure first screens checkpoints with one seed. It then evaluates the five best checkpoints with three seeds. It selects the smallest tree among candidates within 0.01 percentage points of the best mean gap. Only this selected individual enters the independent gate. The gate accepts a candidate when the one-sided 95% upper bound of its validation delta is at most 0.10 percentage points.

The main scientific analysis uses the selected candidate from each GP run. It retains all three GP runs. It does not select the best GP seed. The gate-deployed policy is a separate operational quantity. It falls back to baseline ACO when a candidate fails the gate.

F. Statistical Analysis

The primary response is the reference gap in (3). We also record worst-decile conditional value at risk, anytime gap area

TABLE V
METHODS, CONTROLLED DIFFERENCES, AND CURRENT RESULT STATUS

Method	Transition integration	Pheromone integration	Terminal/function profile	Main purpose	Status
Baseline ACO	None	None	None	Original AS, ACS, or MMAS	Complete
Legacy-GP	Full replacement	None	Legacy	Prior GP-ACO structure under the current budget	Pending
Matched-Replace	Full replacement	None	Full/F1	Replacement control for RQ2	Pending
TR-RGP	Residual	None	Full/F1	Independent transition-only control	Pending
PH-RGP	None	Budget residual	Full/F1	Independent pheromone-only control	Pending
RMTGP Core-F0	Residual	Budget residual	Core/F0	Factorial terminal/function control	Pending
RMTGP Core-F1	Residual	Budget residual	Core/F1	Factorial terminal/function control	Pending
RMTGP Full-F0	Residual	Budget residual	Full/F0	Factorial terminal/function control	Pending
RMTGP Full-F1	Residual	Budget residual	Full/F1	Proposed main method	Complete for uniform tests
Drop and shuffle	Post-hoc intervention	Post-hoc intervention	Frozen Full/F1 trees	Coadaptation analysis for RQ5	Pending

TABLE VI
COMMON EXPERIMENTAL PARAMETERS

Parameter	Value
ACO ants	32
ACO iterations	500
Candidate-list size	20
α, β	1, 2
γ_{tr}, γ_{ph}	1/3, 1/3
Local search	Disabled
GP population	100
GP generations	50
Crossover / mutation / reproduction	0.80/0.15/0.05
Elites / tournament size	10/4
Initial depth / maximum depth	2–4/5
Total effective node budget	31
Checkpoint interval / retained count	5/5
Training instances per generation	32
Selection / gate instances	32/32
Screening / final validation seeds	1/3
Test ACO seeds per instance	3
Independent GP runs per variant	3
CPU threads / process count	16/1
CUDA block threads	32
Search / final score precision	FP32 / FP64

under the curve, best iteration, wall time, and win/tie/loss rates. A win means that the learned method has a smaller gap than its paired baseline.

The three ACO seeds are first averaged within an instance. The main statistical block is then GP-run by instance. We use a paired Wilcoxon signed-rank test [13]. Holm correction is applied within each ACO variant and RQ family [14]. We also use a hierarchical bootstrap [15]. It resamples GP runs, then instances, then ACO seeds. Each interval uses 10,000 replicates.

Only three GP runs are available in this pilot. This count is too small for a final claim about evolutionary-run uncertainty. The confidence intervals and tests are therefore descriptive. A confirmatory study will use 30 independent GP runs.

V. EFFICIENT IMPLEMENTATION

A. Batched Evaluation Architecture

The nested fitness is expensive. One population contains 100 individuals. Each individual invokes ACO on 32 instances. Each ACO invocation constructs 32×500 tours. A direct Python loop over individuals, instances, ants, and iterations is not practical.

The implementation uses NumPy, Numba, PyTorch, DEAP, and CuPy [11], [16]–[19]. The CPU backend forms a population-by-instance task matrix. Numba compiles the inner ACO loops. The GPU backend goes further. It compiles each typed GP tree to a compact postfix program. It removes structurally and behaviorally duplicate programs. It then forms a program-by-instance task grid.

One CUDA block owns one program–instance task. Thirty-two threads match the 32 ants. The block performs all 500 ACO iterations. It evaluates both GP programs inside the kernel. It also performs candidate fallback, ACS local-update multiplicities, MMAS bounds, and MMAS restart logic. Problem tensors and terminal-independent features remain resident on the device. A counter-based random generator makes task partitioning deterministic.

The GPU searches in FP32. It returns the best tour. The host recomputes that tour length from the original float64 distance matrix. The final fitness and all paper tables therefore use CPU FP64 tour scoring.

B. Parallel Scheduling

One GP run uses one process and one visible GPU. Population-level and instance-level work are fused inside the kernel. This design avoids multiple Python processes competing for one GPU. Independent GP runs are the main multi-GPU unit. They are assigned to different devices. This campaign mode maximizes aggregate throughput.

A dual-shard mode can split one generation over two GPUs. It reduces the latency of one run. Its scaling is weaker because each shard is smaller. The registered scheduler therefore uses

TABLE VII
FORMAL ACCELERATOR BENCHMARK

Mode	Time (s)	Tours/s	CPU16 ratio
CPU8	397.422	101,776	0.766
CPU16	304.325	132,911	1.000
GPU0	20.807	1,943,957	14.626
GPU1	20.838	1,941,110	14.605
Dual shard	12.776	3,165,928	23.820
Campaign [†]	21.101	3,833,840	28.9

[†]Two concurrent workloads; ratio uses aggregate throughput.

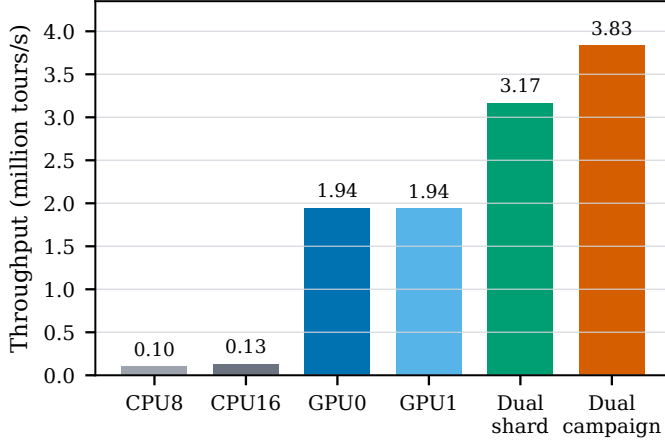


Fig. 2. Throughput on the formal accelerator benchmark. Campaign throughput is aggregated over two concurrent workloads.

independent runs when enough runs are available. CPU16 remains the deterministic fallback and the FP64 audit backend.

C. Formal Acceleration Benchmark

The formal benchmark used an Intel Xeon W-2245 CPU and two 20-GiB NVIDIA RTX 4000 Ada Generation GPUs. The workload used ACS, 100 requested individuals, 79 behaviorally unique programs, 16 TSP50 instances, 16 TSP100 instances, 32 ants, and 500 ACO iterations. One workload constructed 40,448,000 tours. Each mode used one cold run and three warm runs. Table VII reports the warm median.

Single-run dual-GPU scaling is $20.807/12.776 = 1.629$. Concurrent campaign scaling is $2(20.807)/21.101 = 1.972$. The first value is below the registered 1.7 scaling gate. The second value is above the registered 1.8 campaign gate. This result supports task-level multi-GPU scheduling.

Average GPU utilization was about 99%. Peak device memory was about 286 MiB. About 99.8% of warm time was inside the CUDA kernel. The workload was compute-bound. Additional memory caching is therefore unlikely to be the main source of future speedup.

The numerical quality audit used TSP50 and TSP100. It used 128 instances per size and three paired ACO seeds. Let $\Delta_{\text{num}} = g_{\text{GPU}} - g_{\text{CPU}}$. The pooled mean was -0.003445 percentage points. The one-sided 95% upper bound was 0.000868 percentage points. This value is below the registered 0.10 threshold.

TABLE VIII
OBSERVED GENERATION TIME ACROSS THREE GP RUNS

Variant	Mean $\pm$ SD (s)	Run range (s)
AS	34.50 ± 2.97	31.13–36.78
ACS	26.88 ± 4.43	21.97–30.57
MMAS	22.14 ± 3.34	18.31–24.41

The observed mean generation times are shown in Table VIII. They include all unique population evaluations for one generation. They do not include final test evaluation.

VI. RESULTS AND ANALYSIS

A. RQ1: Full-F1 Versus Baseline ACO

Table IX reports the completed selected-candidate results. The baseline column contains the original ACO gap. The Full-F1 column contains the learned-method gap. The confidence interval is for Δg . Negative values favor Full-F1.

All 12 mean differences are negative. All 12 bootstrap intervals are below zero. The smallest observed reduction is 0.0736 percentage points for ACS on TSP50. The largest reduction is 7.8461 percentage points for AS on TSP500. The Holm-adjusted Wilcoxon values are at most 9.004×10^{-17} . These test values describe the current blocks. They do not replace replication over more GP runs.

AS improves from 1.8803% to 1.7479% on TSP50. It improves from 5.8952% to 4.2154% on TSP100. Its reductions are larger on TSP500 and TSP1000. They are 7.8461 and 7.3135 percentage points.

ACS has smaller gains at the two smaller sizes. Its reduction is 0.2918 percentage points on TSP100. It reaches 3.2242 percentage points on TSP500. One of the three ACS candidates did not pass the independent deployment gate. The selected-candidate table retains that run because the analysis must not remove an unfavorable GP replicate. The operational gate-deployed result is therefore weaker than the selected-candidate result for ACS.

MMAS improves from 4.1088% to 1.8106% on TSP100. Its reductions are 6.4522 and 6.6314 percentage points on TSP500 and TSP1000. Its TSP100 win rate is 99.2%. Its TSP500 and TSP1000 win rates are 100% in the current blocks.

Fig. 3 shows noisy training curves. This behavior is expected because every generation uses a new batch. The validation curves are also not monotonic. The selected checkpoint can therefore differ from the last generation. The independent selection procedure is necessary.

Answer to RQ1: Full-F1 has a lower mean reference gap than the corresponding original ACO in every completed pilot comparison. This is evidence for effectiveness. It is not yet evidence that the residual interface, two-tree structure, or Full/F1 primitive profile is the cause.

B. RQ2: Residual Versus Full Replacement

Pending. This question requires a controlled transition-only comparison. The primary contrast is

$$\Delta_{\text{RQ2}} = g_{\text{TR-RGP}} - g_{\text{Matched-Replace}}. \quad (28)$$

TABLE IX

SELECTED-CANDIDATE RESULTS ON COMPLETED UNIFORM TEST SETS. CI DENOTES THE HIERARCHICAL BOOTSTRAP 95% INTERVAL OF Δg . W/T/L IS REPORTED IN PERCENT.

ACO	n	Baseline gap (%)	Full-F1 gap (%)	Δg (pp)	95% CI (pp)	W/T/L
AS	50	1.8803	1.7479	-0.1324	[-0.2076, -0.0718]	54.7/2.8/42.4
AS	100	5.8952	4.2154	-1.6799	[-1.9680, -1.4739]	92.1/0.0/7.9
AS	500	22.9874	15.1413	-7.8461	[-8.3979, -7.3727]	100.0/0.0/0.0
AS	1000	30.4224	23.1088	-7.3135	[-8.5940, -6.1418]	100.0/0.0/0.0
ACS	50	0.7326	0.6589	-0.0736	[-0.1050, -0.0432]	46.6/17.6/35.9
ACS	100	1.9331	1.6412	-0.2918	[-0.3954, -0.2091]	64.7/0.0/35.3
ACS	500	16.1641	12.9399	-3.2242	[-4.5698, -1.3932]	93.8/0.0/6.2
ACS	1000	21.6293	19.2041	-2.4252	[-3.3961, -0.8317]	94.8/0.0/5.2
MMAS	50	0.5941	0.4727	-0.1215	[-0.1549, -0.0820]	52.3/22.4/25.2
MMAS	100	4.1088	1.8106	-2.2982	[-2.4240, -2.1223]	99.2/0.0/0.8
MMAS	500	20.6291	14.1769	-6.4522	[-8.1652, -4.7067]	100.0/0.0/0.0
MMAS	1000	25.4885	18.8571	-6.6314	[-8.3746, -4.8120]	100.0/0.0/0.0

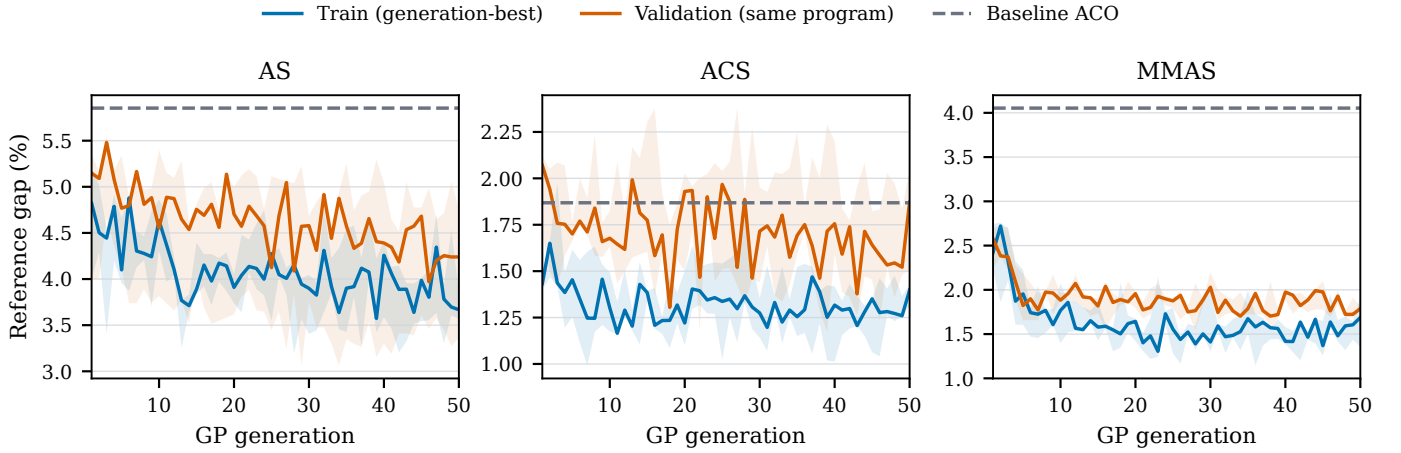


Fig. 3. TSP100 training and validation curves. Lines are medians over three independent GP runs. Shaded regions show the minimum and maximum candidate values. A new training batch is used in every generation.

Both methods use the Full terminals and F1. Both use one transition tree and the same total node budget. They differ only in the integration interface.

Legacy-GP provides a second replacement baseline. It uses the terminal and function design of the previous study. It cannot isolate the residual effect because several factors change at once.

Current answer to RQ2: No conclusion is available. The RQ1 result cannot answer this question.

C. RQ3: Two Trees Versus One Tree

Pending. The primary comparison uses Full-F1, TR-RGP, and PH-RGP. All methods use a total effective node budget of 31. The comparison therefore controls total syntax capacity. It does not force each active tree to use the same number of nodes.

A second analysis uses budgets of 31 and 62. It will test whether a two-tree advantage remains when capacity is increased. The independent one-tree runs are the main controls. Dropping one tree from a trained pair is only a post-hoc mechanism test.

Current answer to RQ3: No conclusion is available. The completed Full-F1 versus ACO comparison does not establish a two-tree advantage.

D. RQ4: Terminal and Function Profiles

Pending. The factorial experiment contains Core-F0, Core-F1, Full-F0, and Full-F1. The terminal main effect is the average Full minus Core contrast. The function main effect is the average F1 minus F0 contrast. The interaction tests whether the F1 effect changes with the terminal profile.

The Core profile contains four transition terminals and four pheromone terminals. The Full profile adds progress, entropy, stagnation, and colony-frequency information. F1 adds MIN, MAX, and ABS. This design can detect unnecessary primitives. It can also detect a dependence between the richer functions and richer terminals.

Current answer to RQ4: No conclusion is available. The present terminal and function sets are technically valid. Their empirical necessity remains untested.

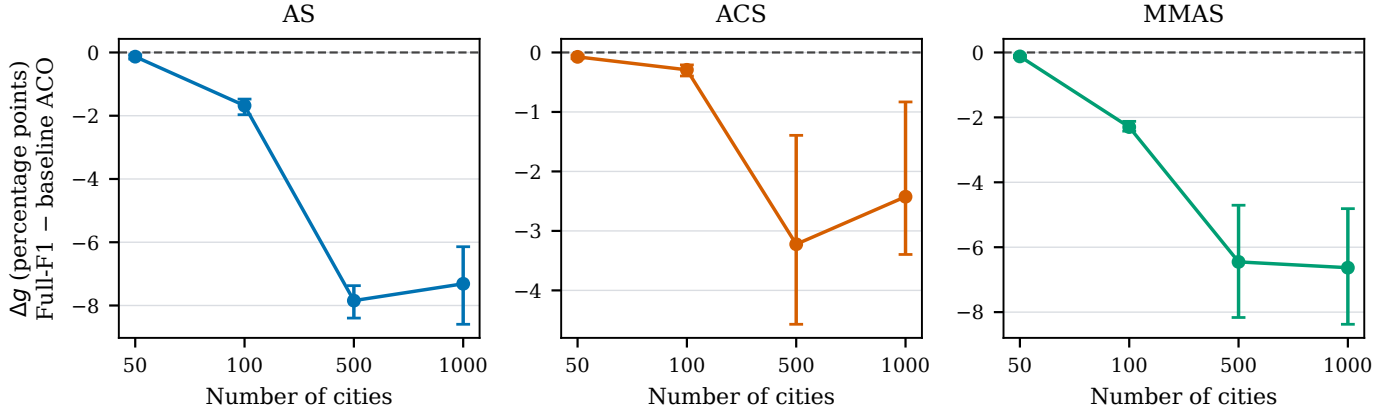


Fig. 4. Full-F1 minus baseline ACO reference gap. Error bars are hierarchical bootstrap 95% intervals. Negative values favor Full-F1.

E. RQ5: Joint Coadaptation

Pending. The mechanism analysis will evaluate three interventions. The first replaces the transition tree with zero. The second replaces the pheromone tree with zero. The third shuffles transition and pheromone trees across independent GP runs. A loss after shuffling would indicate pair-specific coadaptation. It would not by itself prove that joint training is better than independent single-tree training.

Current answer to RQ5: No conclusion is available.

F. RQ6: Generalization

The completed results support scale transfer on uniform Euclidean TSP. Training and validation use only TSP100. TSP50, TSP500, and TSP1000 are unseen sizes. Full-F1 has a negative mean Δg for all three variants at every unseen size. The absolute reduction is generally larger at TSP500 and TSP1000.

This result does not establish distribution transfer. Clustered TSP500, Gaussian TSP500, and TSPLIB results are pending. No terminal, function, or mechanism claim will be made from an out-of-distribution result alone.

Answer to RQ6: The current pilot supports transfer across uniform problem sizes. Distribution transfer remains unanswered.

G. RQ7: Computational Efficiency

The fused evaluator gives a 14.626-fold single-GPU speedup over CPU16 on the registered workload. Dual sharding gives a 23.820-fold speedup over CPU16. Its scaling over one GPU is 1.629. Concurrent two-run campaign execution gives a 1.972-fold throughput scaling over one GPU.

Answer to RQ7: Population-by-instance fusion makes full-budget GP training practical. Independent runs are the preferred multi-GPU unit. The measured ratios apply to the registered RTX 4000 Ada workload. They must not be transferred to another GPU model without a new benchmark.

VII. LIMITATIONS AND THREATS TO VALIDITY

The main limitation is the number of GP runs. Three runs can reveal major failures. They cannot characterize the full

variance of evolution. The current intervals contain substantial instance-level information but limited run-level information.

Training uses only uniform TSP100. The present result is therefore a transfer study from one size and one distribution. The method has not yet been tested with local search. The conclusions apply only to AS, synchronous ACS, and MMAS under the reported parameters.

The reference tours are valid. They do not all have attached global optimality certificates. Reference gap can therefore be negative in principle. The metric remains suitable for paired comparison because every method uses the same reference.

GPU search uses FP32. CPU and GPU can follow different stochastic trajectories. The final length calculation is FP64, and the registered numerical non-inferiority test passes. This result does not imply bitwise equivalence.

The current RQ1 data cannot support a causal explanation. Residual versus replacement, single versus dual tree, primitive profiles, and coadaptation require their registered controls. Those results are deliberately marked as pending.

VIII. CONCLUSION

This paper introduced RMTGP-ACO. The method evolves two strongly typed residual programs. One program changes state transition desirability. The other redistributes pheromone deposits. The interfaces preserve useful ACO invariants. The two trees also share a fixed total node budget.

The completed pilot uses three independent GP runs for AS, ACS, and MMAS. Full-F1 has a lower mean reference gap than baseline ACO in every completed uniform test. The result holds from TSP50 to TSP1000. A fused CUDA evaluator reduces the cost of the nested GP-ACO computation.

These findings support further evaluation. They do not yet show that residual integration is better than replacement. They also do not yet show that two trees are better than one tree. The pending ablation and capacity experiments are required for those conclusions.

REFERENCES

- [1] M. Dorigo, V. Maniezzo, and A. Colomi, "Ant system: Optimization by a colony of cooperating agents," *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, vol. 26, no. 1, pp. 29–41, 1996.

- [2] M. Dorigo and T. Stützle, *Ant Colony Optimization*. Cambridge, MA, USA: MIT Press, 2004.
- [3] J. R. Koza, *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. Cambridge, MA, USA: MIT Press, 1992.
- [4] R. Poli, W. B. Langdon, and N. F. McPhee, *A Field Guide to Genetic Programming*. Lulu Enterprises, 2008, with contributions by J. R. Koza.
- [5] B.-C. Lin, Y. Mei, and M. Zhang, “Automated design of state transition rules in ant colony optimization by genetic programming: A comprehensive investigation,” *Memetic Computing*, vol. 17, p. 2, 2025.
- [6] J. Tavares and F. B. Pereira, “Designing pheromone update strategies with strongly typed genetic programming,” in *Genetic Programming*, ser. Lecture Notes in Computer Science, vol. 6621. Springer, 2011, pp. 85–96.
- [7] M. Dorigo and L. M. Gambardella, “Ant colony system: A cooperative learning approach to the traveling salesman problem,” *IEEE Transactions on Evolutionary Computation*, vol. 1, no. 1, pp. 53–66, 1997.
- [8] T. Stützle and H. H. Hoos, “MAX–MIN ant system,” *Future Generation Computer Systems*, vol. 16, no. 8, pp. 889–914, 2000.
- [9] T. Stützle, “ACOTSP, version 1.0,” Software package, 2004, version 1.03 used as the algorithmic reference.
- [10] D. J. Montana, “Strongly typed genetic programming,” *Evolutionary Computation*, vol. 3, no. 2, pp. 199–230, 1995.
- [11] F.-A. Fortin, F.-M. D. Rainville, M.-A. Gardner, M. Parizeau, and C. Gagné, “DEAP: Evolutionary algorithms made easy,” *Journal of Machine Learning Research*, vol. 13, pp. 2171–2175, 2012.
- [12] G. Reinelt, “TSPLIB: A traveling salesman problem library,” *ORSA Journal on Computing*, vol. 3, no. 4, pp. 376–384, 1991.
- [13] F. Wilcoxon, “Individual comparisons by ranking methods,” *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.
- [14] S. Holm, “A simple sequentially rejective multiple test procedure,” *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.
- [15] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman and Hall/CRC, 1994.
- [16] C. R. Harris, K. J. Millman, S. J. van der Walt *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.
- [17] S. K. Lam, A. Pitrou, and S. Seibert, “Numba: A LLVM-based Python JIT compiler,” in *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*. ACM, 2015, pp. 1–6.
- [18] A. Paszke, S. Gross, F. Massa *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019, pp. 8024–8035.
- [19] R. Okuta, Y. Unno, D. Nishino, S. Hido, and C. Loomis, “CuPy: A NumPy-compatible library for NVIDIA GPU calculations,” in *Proceedings of the Workshop on Machine Learning Systems at NeurIPS*, 2017.